

HDOC – Experiment 16

Question

Write a menu-driven C program using switch-case to perform the following operations on a positive integer using separate user-defined functions (void return type and simple function call):

- 1. Check whether digits are in increasing order from left to right.**
- 2. Check whether digits are in decreasing order from left to right.**
- 3. Find the largest digit and the smallest digit.**
- 4. Find the second largest digit and second smallest digit.**

❖ Understanding problem

Requirements

- One separate user-defined function for every option.
- Every function must have void return type.
- Functions must be called without arguments, for example:

increasing();
- Each function itself takes the number as input, processes it, and displays the result.
- Use switch-case for the menu.

❖ Algorithm

- **Main Program Algorithm**

1. Start.
2. Display the menu:
 - 1. Increasing order
 - 2. Decreasing order
 - 3. Largest and smallest digit
 - 4. Second largest and second smallest digit
3. Input the user's choice.
4. Use switch(choice).
5. If choice is 1, call increasing().
6. If choice is 2, call decreasing().
7. If choice is 3, call largestSmallest().
8. If choice is 4, call secondLargestSmallest().
9. For any other choice, display "Invalid choice".
10. Stop.

- **Algorithm for increasing()**

1. Start.
2. Input a positive integer num.
3. Check whether num is positive.
4. Set temp = num.
5. Take the rightmost digit as prev.
6. Remove the rightmost digit from temp.
7. Repeat while temp > 0:
 - Find the next digit current.
 - If current >= prev, digits are not in increasing order.
 - Otherwise, set prev = current.
 - Remove current from temp.
8. Display the result.
9. Stop.

- **Algorithm for decreasing()**

1. Start.
2. Input a positive integer num.
3. Check whether num is positive.
4. Set temp = num.
5. Take the rightmost digit as prev.
6. Remove the rightmost digit from temp.
7. Repeat while temp > 0:
 - Find current = temp % 10.
 - If current <= prev, digits are not in decreasing order.

- Otherwise, set $prev = current$.
 - Remove the current digit from temp.
8. Display the result.
 9. Stop.

- **Algorithm for largestSmallest()**

1. Start.
2. Input a positive integer num.
3. Check whether num is positive.
4. Initialize $largest = 0$ and $smallest = 9$.
5. Repeat while $num > 0$:
 - Find $digit = num \% 10$.
 - If $digit > largest$, set $largest = digit$.
 - If $digit < smallest$, set $smallest = digit$.
 - Remove the digit using $num = num / 10$.
6. Display largest.
7. Display smallest.
8. Stop.

- **Algorithm for secondLargestSmallest()**

1. Start.
2. Input a positive integer num.
3. Check whether num is positive.
4. Create an array `present[10]` and initialize all elements to 0.
5. Extract each digit of the number.

6. Mark that digit as present using `present[digit] = 1`.
7. Search from 0 to 9.
8. The first present digit is the smallest.
9. The next present digit is the second smallest.
10. Search from 9 down to 0.
11. The first present digit is the largest.
12. The next present digit is the second largest.
13. If fewer than two distinct digits exist, display an appropriate message.
14. Otherwise, display the second largest and second smallest digits.
15. Stop.

❖ Test case table

Test case	Menu choice	Input number	Expected outcome	Actual outcome	Result
1	1	12345	Digits are in increasing order	Same	Pass
2	1	12435	Digits are not in increasing order	Same	Pass
3	2	97531	Digits are in decreasing order	Same	Pass
4	2	97351	Digits are not in decreasing order	Same	Pass
5	3	58321	Largest = 8, Smallest = 1	Same	Pass
6	3	90724	Largest = 9, Smallest = 0	Same	Pass
7	4	58321	Second largest = 5, Second smallest = 2	Same	Pass
8	4	92841	Second largest = 8, Second smallest = 2	Same	Pass

9	4	77777	Second largest/smallest do not exist	Same	Pass
10	Any	-25	Invalid input	Same	Pass

❖ Program code in C

```
#include <stdio.h>
```

```
/* Function declarations */
```

```
void increasing();
```

```
void decreasing();
```

```
void largestSmallest();
```

```
void secondLargestSmallest();
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    printf("DIGIT OPERATIONS MENU\n");
```

```
    printf("1. Check increasing order\n");
```

```
    printf("2. Check decreasing order\n");
```

```
    printf("3. Find largest and smallest digit\n");
```

```
    printf("4. Find second largest and second smallest digit\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
switch (choice)
{
    case 1:
        increasing();
        break;

    case 2:
        decreasing();
        break;

    case 3:
        largestSmallest();
        break;

    case 4:
        secondLargestSmallest();
        break;

    default:
        printf("Invalid choice.\n");
}

return 0;
}
```

/* Function to check increasing order */

```
void increasing()
```

```
{  
    int num, temp, prev, current;  
    int flag = 1;  
  
    printf("Enter a positive integer: ");  
    scanf("%d", &num);  
  
    if (num <= 0)  
    {  
        printf("Invalid input. Enter a positive integer.\n");  
        return;  
    }  
  
    temp = num;  
  
    prev = temp % 10;  
    temp = temp / 10;  
  
    while (temp > 0)  
    {  
        current = temp % 10;  
  
        if (current >= prev)  
        {  
            flag = 0;  
            break;  
        }  
    }
```

```
    prev = current;
    temp = temp / 10;
}
```

```
if (flag == 1)
    printf("Digits are in increasing order.\n");
else
    printf("Digits are not in increasing order.\n");
}
```

/* Function to check decreasing order */

```
void decreasing()
{
    int num, temp, prev, current;
    int flag = 1;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num <= 0)
    {
        printf("Invalid input. Enter a positive integer.\n");
        return;
    }

    temp = num;

    prev = temp % 10;
```

```
temp = temp / 10;

while (temp > 0)
{
    current = temp % 10;

    if (current <= prev)
    {
        flag = 0;
        break;
    }

    prev = current;
    temp = temp / 10;
}

if (flag == 1)
    printf("Digits are in decreasing order.\n");
else
    printf("Digits are not in decreasing order.\n");
}
```

/* Function to find largest and smallest digit */

```
void largestSmallest()
{
    int num, digit;
    int largest = 0;
    int smallest = 9;
```

```
printf("Enter a positive integer: ");
scanf("%d", &num);

if (num <= 0)
{
    printf("Invalid input. Enter a positive integer.\n");
    return;
}

while (num > 0)
{
    digit = num % 10;

    if (digit > largest)
        largest = digit;

    if (digit < smallest)
        smallest = digit;

    num = num / 10;
}

printf("Largest digit = %d\n", largest);
printf("Smallest digit = %d\n", smallest);
}

/* Function to find second largest and second smallest distinct digits */
```

```

void secondLargestSmallest()
{
    int num, digit, i;

    int present[10] = {0};

    int count = 0;

    int secondSmallest = -1;

    int secondLargest = -1;


    printf("Enter a positive integer: ");

    scanf("%d", &num);


    if (num <= 0)
    {
        printf("Invalid input. Enter a positive integer.\n");

        return;
    }


/* Record which digits are present */

    while (num > 0)
    {
        digit = num % 10;

        present[digit] = 1;

        num = num / 10;
    }


/* Find second smallest distinct digit */

    count = 0;

```

```
for (i = 0; i <= 9; i++)
{
    if (present[i] == 1)
    {
        count++;

        if (count == 2)
        {
            secondSmallest = i;
            break;
        }
    }
}
```

/* Find second largest distinct digit */

```
count = 0;
```

```
for (i = 9; i >= 0; i--)
{
    if (present[i] == 1)
    {
        count++;

        if (count == 2)
        {
            secondLargest = i;
            break;
        }
    }
}
```

```

    }
}

if (secondSmallest == -1 || secondLargest == -1)
{
    printf("Second largest and second smallest digits do not exist.\n");
}
else
{
    printf("Second largest digit = %d\n", secondLargest);
    printf("Second smallest digit = %d\n", secondSmallest);
}
}

```

❖ Compile, Execute and Evaluate

Test case 1

```

(kali㉿kali)-[~/Desktop/exp16]
$ ./check
DIGIT OPERATIONS MENU
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12345
Digits are in increasing order.

```

Test case 2

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 1  
Enter a positive integer: 12435  
Digits are not in increasing order.
```

Test case 3

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 2  
Enter a positive integer: 97531  
Digits are in decreasing order.
```

Test case 4

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 2  
Enter a positive integer: 97351  
Digits are not in decreasing order.
```

Test case 5

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 3  
Enter a positive integer: 58321  
Largest digit = 8  
Smallest digit = 1
```

Test case 6

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 3  
Enter a positive integer: 90724  
Largest digit = 9  
Smallest digit = 0
```

Test case 7

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 4  
Enter a positive integer: 58321  
Second largest digit = 5  
Second smallest digit = 2
```

Test case 8

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 4  
Enter a positive integer: 92841  
Second largest digit = 8  
Second smallest digit = 2
```

Test case 9

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 4  
Enter a positive integer: 77777  
Second largest and second smallest digits do not exist.
```

Test case 10

```
(kali㉿kali)-[~/Desktop/exp16]  
$ ./check  
DIGIT OPERATIONS MENU  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice: 3  
Enter a positive integer: -25  
Invalid input. Enter a positive integer.
```